TÀI LIỆU THỰC HÀNH CHI TIẾT

Chủ đề: Deploy ứng dụng ReactJS từ thủ công đến CI/CD

Mục tiêu chung

Sau khi hoàn thành 3 bài lab này, học viên sẽ:

- Hiểu quy trình build ứng dụng React thành file tĩnh và phục vụ bằng Nginx.
- Biết cách đưa ứng dụng lên VPS theo cách thủ công.
- Biết cách đóng gói ứng dụng bằng Docker với multi-stage build để tối ưu kích thước image.
- Biết cách tự động hóa build và deploy bằng GitHub Actions.
- Làm quen với quy trình rollback khi phiên bản mới gặp lỗi.

Ứng dụng mẫu dùng trong toàn bộ bài lab

Toàn bộ bài thực hành sử dụng ứng dụng **Simple ReactJS** đã được chuẩn bị sẵn:

- Viết bằng **React 19 + Vite**
- Được phục vụ bởi **Nginx** trên cổng **80**
- Source code được lưu trên **GitHub** cá nhân
- Hệ điều hành máy chủ là **Ubuntu**

Cấu trúc thư mục dự án:

```
simple-reactjs/
├── src/
│   ├── App.jsx
│   ├── main.jsx
│   └── components/
├── public/
├── index.html
├── package.json
├── vite.config.js
├── nginx.conf
├── Dockerfile
└── docker-compose.yml
```

Điểm khác biệt quan trọng so với Node.js:

> Ứng dụng React **không chạy trực tiếp** trên server. Trước khi deploy, cần chạy lệnh `npm run build` để biên dịch toàn bộ source code thành các file tĩnh (HTML, JS, CSS) đặt trong thư mục `dist/`. Nginx sau đó sẽ phục vụ các file này.

PHẦN 1. CHUẨN BỊ MÔI TRƯỜNG

1.1. Yêu cầu trước khi bắt đầu

Mỗi học viên cần chuẩn bị sẵn:

Trên máy cá nhân:

- Một máy tính có kết nối Internet
- Có cài Terminal:
  - Windows: PowerShell hoặc Windows Terminal
  - macOS/Linux: Terminal mặc định
- Có tài khoản GitHub
- Có tài khoản Docker Hub
- Có cài Git
- Có cài Node.js 18 trở lên:

  ```
  node -v
  npm -v
  ```

- Có cài Docker Desktop (dùng từ Lab 2 trở đi)

Trên máy chủ:

- Một VPS Ubuntu
- Biết các thông tin:
  - IP VPS
  - username đăng nhập: `root`
  - mật khẩu hoặc SSH Key

1.2. Kiểm tra source code

Clone repo về máy (nếu chưa có):

```
git clone https://github.com/gianglt-dau/simple-reactjs.git
cd simple-reactjs
```

Cài dependency và kiểm tra ứng dụng chạy được trên máy local:

```
npm install
npm run dev
```

Mở trình duyệt tại `http://localhost:5173`.

> **Kết quả mong đợi:** Ứng dụng React hiển thị bình thường trên máy local.

LAB 1. DEPLOY THỦ CÔNG LÊN VPS VỚI NGINX

2.1. Mục tiêu

Sau lab này, học viên sẽ:

- Biết build ứng dụng React thành file tĩnh
- Biết cài và cấu hình Nginx trên VPS
- Biết clone source code từ GitHub lên VPS bằng `git clone`
- Biết cấu hình Nginx để phục vụ Single Page Application (SPA)
- Hiểu tại sao React Router cần cấu hình đặc biệt trên server

---

2.2. Bước 1: SSH vào máy chủ

Trong **Terminal** (PowerShell trên Windows, Terminal trên macOS/Linux), đăng nhập vào VPS:

```
ssh <username>@<IP_VPS>
```

> **Kết quả mong đợi:** Đăng nhập thành công vào máy chủ:
>
> ```
> username@ubuntu-server:~$
> ```

> **Lỗi thường gặp:**
>
> - `Permission denied`: sai mật khẩu
> - `Connection refused`: dịch vụ SSH chưa chạy (chạy `sudo systemctl start ssh`)
> - `Host key verification failed`: chạy `ssh-keyscan <IP_VPS> >> ~/.ssh/known_hosts` trên máy local trước

2.3. Bước 2: Cập nhật hệ điều hành

```
sudo apt update && sudo apt upgrade -y
```

> **Kết quả mong đợi:** Cập nhật thành công, không có lỗi nghiêm trọng.

2.4. Bước 3: Cài đặt Git và Nginx

```
sudo apt install -y git nginx
```

Kiểm tra Nginx đang chạy:

```
sudo systemctl status nginx
```

> **Kết quả mong đợi:** Trạng thái hiển thị `active (running)`.

Kiểm tra bằng trình duyệt, truy cập `http://<IP_VPS>`.

> **Kết quả mong đợi:** Thấy trang mặc định **"Welcome to nginx!"**

2.5. Bước 4: Cài đặt Node.js 22

Ubuntu mặc định cài Node.js phiên bản cũ. Cần thêm repo của NodeSource để cài Node.js 22:

```
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
```

Kiểm tra phiên bản:

```
node -v
npm -v
```

> **Kết quả mong đợi:** `node -v` hiển thị `v22.x.x`

> **Tại sao không dùng `apt install nodejs` trực tiếp?** Repo mặc định của Ubuntu thường chứa Node.js phiên bản cũ (12 hoặc 18). Dùng script của NodeSource để đảm bảo cài đúng Node.js 22, tương thích với dự án.

2.6. Bước 5: Clone source code từ GitHub

Trong phiên SSH vào máy chủ, clone repo về:

```
cd ~
git clone https://github.com/gianglt-dau/simple-reactjs.git
cd simple-reactjs
```

> **Kết quả mong đợi:** Thư mục `~/simple-reactjs/` được tạo với đầy đủ source code.

```
ls
```

> **Kết quả mong đợi:** Thấy các file `package.json`, `index.html`, `vite.config.js`, `Dockerfile`...

2.7. Bước 6: Tạo thư mục chứa file tĩnh

Thư mục `/var/www` thuộc sở hữu của `root`, cần dùng `sudo` để tạo và cấp quyền:

```
sudo mkdir -p /var/www/simple-reactjs
sudo chown $USER:$USER /var/www/simple-reactjs
```

Giải thích:

- `sudo mkdir`: tạo thư mục với quyền root
- `sudo chown $USER:$USER`: chuyển quyền sở hữu về user hiện tại, để có thể copy file vào mà không cần sudo

---

2.8. Bước 7: Build ứng dụng trên máy chủ

Di chuyển vào thư mục source vừa clone:

```
cd ~/simple-reactjs
npm install
npm run build
```

Giải thích: `npm run build` kích hoạt Vite biên dịch toàn bộ source code React thành file tĩnh trong thư mục `dist/`.

Kiểm tra kết quả build:

```
ls dist/
```

> **Kết quả mong đợi:**
>
> ```
> assets  index.html
> ```

Copy file build vào thư mục web:

```
cp -r dist/* /var/www/simple-reactjs/
```

Kiểm tra lại:

```
ls /var/www/simple-reactjs/
```

> **Kết quả mong đợi:**
>
> ```
> assets  index.html
> ```

2.9. Bước 8: Cấu hình Nginx cho ứng dụng SPA

Tạo file cấu hình mới cho Nginx:

```
sudo nano /etc/nginx/sites-available/simple-reactjs
```

Nhập nội dung sau:

```
server {
    listen 80;
    server_name localhost;
    root /var/www/simple-reactjs;
    index index.html;

    # Handle React Router (SPA fallback)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets
    location ~* \.(?:js|css|png|jpg|jpeg|gif|ico|svg|woff2?)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
}
```

Nhấn `Ctrl + O` để lưu, `Ctrl + X` để thoát.

Giải thích dòng quan trọng:

```
try_files $uri $uri/ /index.html;
```

> Khi người dùng truy cập `/about` hoặc `/dashboard`, Nginx tìm file tương ứng trên đĩa. Nếu không có, nó phục vụ `index.html` để React Router tự xử lý routing phía client. Thiếu dòng này, người dùng sẽ gặp lỗi **404** khi refresh trang hoặc truy cập trực tiếp một URL.

2.10. Bước 9: Kích hoạt site và khởi động lại Nginx

Tạo symlink để kích hoạt cấu hình:

```
sudo ln -s /etc/nginx/sites-available/simple-reactjs /etc/nginx/sites-enabled/
```

Vô hiệu hóa site mặc định của Nginx (tránh xung đột):

```
sudo rm /etc/nginx/sites-enabled/default
```

Kiểm tra cú pháp cấu hình:

```
sudo nginx -t
```

> **Kết quả mong đợi:**
>
> ```
> nginx: configuration file /etc/nginx/nginx.conf syntax is ok
> nginx: configuration file /etc/nginx/nginx.conf test is successful
> ```

Khởi động lại Nginx:

```
sudo systemctl reload nginx
```

2.11. Bước 10: Kiểm tra kết quả

Mở trình duyệt và truy cập `http://<IP_VPS>`.

> **Kết quả mong đợi:** Ứng dụng React hiển thị bình thường.

2.11b. Xử lý khi cổng 80 đã bị chiếm

Nếu sau khi `sudo systemctl reload nginx` ứng dụng không hiển thị, hoặc `sudo nginx -t` báo lỗi về cổng, rất có thể cổng 80 đang bị một tiến trình khác (thường là Apache2) chiếm dụng.

Bước 1: Kiểm tra tiến trình nào đang dùng cổng 80

```
sudo ss -tlnp | grep :80
```

Ví dụ kết quả cho thấy Apache2 đang chiếm cổng:

```
LISTEN  0  511  0.0.0.0:80  0.0.0.0:*  users:(("apache2",pid=1234,...))
```

Cách xử lý 1: Dừng service đang chiếm cổng 80 (nếu được phép)

```
sudo systemctl stop apache2
sudo systemctl disable apache2   # Tắt luôn khi khởi động
sudo systemctl reload nginx
```

> **Kết quả mong đợi:** Nginx chiếm lại cổng 80. Truy cập `http://<IP_VPS>` hoạt động bình thường.

Cách xử lý 2: Đổi Nginx sang cổng khác (ví dụ 8080)

Nếu không thể dừng service kia, chỉnh file cấu hình Nginx để dùng cổng khác:

```
sudo nano /etc/nginx/sites-available/simple-reactjs
```

Thay `listen 80;` thành `listen 8080;`:

```
server {
    listen 8080;
    server_name localhost;
    root /var/www/simple-reactjs;
    index index.html;

    location / {
        try_files $uri $uri/ /index.html;
    }
    ...
}
```

Kiểm tra cú pháp và reload:

```
sudo nginx -t
sudo systemctl reload nginx
```

Mở cổng 8080 trên firewall (nếu dùng `ufw`):

```
sudo ufw allow 8080
```

Truy cập ứng dụng tại `http://<IP_VPS>:8080`.

> **Lưu ý:** Nếu dùng cloud provider (AWS, GCP, Azure, DigitalOcean...), nhớ mở cổng 8080 trong **Security Group / Firewall Rules** của cloud console, không chỉ trên `ufw`.

2.12. Cập nhật ứng dụng (khi có thay đổi code)

SSH vào máy chủ và kéo code mới từ GitHub:

```
cd ~/simple-reactjs
git pull origin main
npm run build
cp -r dist/* /var/www/simple-reactjs/
```

Không cần restart Nginx — Nginx sẽ tự phục vụ file mới.

2.13. Kết luận Lab 1

- Ứng dụng React phải được **build trước** thành file tĩnh, không thể chạy trực tiếp source code JSX
- Nginx phục vụ file tĩnh nhanh và hiệu quả, không cần Node.js trên server
- Cần cấu hình `try_files $uri $uri/ /index.html` để React Router hoạt động đúng
- Hạn chế: mỗi lần cập nhật phải build và upload thủ công

LAB 2. ĐÓNG GÓI BẰNG DOCKER VỚI MULTI-STAGE BUILD

3.1. Mục tiêu

Sau lab này, học viên sẽ:

- Hiểu khái niệm multi-stage build trong Docker
- Biết đọc và phân tích Dockerfile của ứng dụng React
- Biết build Docker image và push lên Docker Hub
- Biết chạy container trên VPS bằng docker-compose
- Hiểu lợi ích của "build ở local, run ở server"

3.2. Ý tưởng multi-stage build

Ở Lab 1, bạn cần cài Node.js trên máy local để build, rồi mới có file tĩnh để upload. Có một vấn đề: nếu người khác muốn build, họ cũng phải cài cùng phiên bản Node.js.

Multi-stage build trong Docker giải quyết điều này:

- **Stage 1 (builder):** Dùng image Node.js để cài dependency và build ứng dụng React
- **Stage 2 (production):** Chỉ copy kết quả từ Stage 1 vào image Nginx nhỏ gọn

Kết quả: image production **không chứa Node.js**, chỉ chứa Nginx và file tĩnh — rất nhỏ và an toàn.

3.3. Phân tích Dockerfile của dự án

Mở file `Dockerfile` trong thư mục dự án:

```
# Stage 1: Build
FROM node:22-alpine AS builder

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

# Stage 2: Serve with Nginx
FROM nginx:stable-alpine AS production

# Remove default nginx static assets
RUN rm -rf /usr/share/nginx/html/*

# Copy built assets from builder
COPY --from=builder /app/dist /usr/share/nginx/html

# Copy custom nginx config
COPY nginx.conf /etc/nginx/conf.d/default.conf

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Giải thích từng phần:

Dòng	Nội dung	Giải thích
`FROM node:22-alpine AS builder`	Khai báo Stage 1	Dùng Node.js 22 bản Alpine (gọn nhẹ) để build
`COPY package*.json ./` + `RUN npm ci`	Cài dependency	Copy khai báo thư viện trước để tối ưu Docker cache
`npm ci`	Cài đúng phiên bản	Khác với `npm install`, `npm ci` đảm bảo cài đúng version trong lockfile
`RUN npm run build`	Build React	Tạo ra thư mục `dist/`

Dòng	Nội dung	Giải thích
`FROM nginx:stable-alpine AS production`	Khai báo Stage 2	Image mới, sạch, chỉ có Nginx
`COPY --from=builder /app/dist ...`	Copy kết quả build	Lấy file từ Stage 1, không cần Node.js nữa
`COPY nginx.conf ...`	Cấu hình Nginx	Dùng file nginx.conf tùy chỉnh của dự án
`EXPOSE 80`	Khai báo cổng	Container lắng nghe trên cổng 80

3.4. Phân tích file nginx.conf

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    # Handle React Router (SPA fallback)
    location / {
        try_files $uri $uri/ /index.html;
    }

    # Cache static assets
    location ~* \.(?:js|css|png|jpg|jpeg|gif|ico|svg|woff2?)$ {
        expires 1y;
        add_header Cache-Control "public, immutable";
    }

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";
}
```

> File này được copy vào image ở Stage 2, đảm bảo cấu hình Nginx giống hệt nhau ở mọi môi trường.

3.5. Bước 1: Đăng nhập Docker Hub trên máy local

```
docker login
```

Nhập Docker Hub username và password khi được nhắc.

> **Kết quả mong đợi:** `Login Succeeded`

3.6. Bước 2: Build Docker image

Trong thư mục dự án:

```
docker build -t <your_dockerhub_username>/simple-reactjs:v1.0.0 .
```

> **Lưu ý:** Dấu `.` ở cuối là bắt buộc — đại diện cho thư mục hiện tại.

Quan sát quá trình build:

```
[+] Building 45.2s
 => [builder 1/6] FROM node:22-alpine         ...
 => [builder 2/6] WORKDIR /app                ...
 => [builder 3/6] COPY package*.json ./       ...
 => [builder 4/6] RUN npm ci                  ...
 => [builder 5/6] COPY . .                    ...
 => [builder 6/6] RUN npm run build           ...
 => [production 1/3] FROM nginx:stable-alpine ...
 => [production 2/3] COPY --from=builder ...  ...
 => [production 3/3] COPY nginx.conf ...      ...
```

Kiểm tra image:

```
docker images
```

> **Kết quả mong đợi:** Thấy image `<username>/simple-reactjs v1.0.0` với kích thước khoảng **20–30 MB** (rất nhỏ vì không chứa Node.js).

3.7. Bước 3: Kiểm tra container trên máy local

Chạy thử container trước khi push:

```
docker run -d -p 3000:80 --name test-reactjs <your_dockerhub_username>/simple-
reactjs:v1.0.0
```

Mở trình duyệt tại `http://localhost:3000`.

> **Kết quả mong đợi:** Ứng dụng React hiển thị bình thường.

Dừng và xóa container test:

```
docker stop test-reactjs
```

```
docker rm test-reactjs
```

3.8. Bước 4: Push image lên Docker Hub

```
docker push <your_dockerhub_username>/simple-reactjs:v1.0.0
```

> **Kết quả mong đợi:** Image được tải lên Docker Hub thành công.

Truy cập `https://hub.docker.com/r/<your_dockerhub_username>/simple-reactjs` để xác nhận.

3.9. Bước 5: Chuẩn bị VPS để chạy Docker

SSH vào VPS và cài Docker từ repo chính thức:

Gỡ bản Docker cũ (nếu có)

```
sudo apt remove -y docker.io
```

Cài repo chính thức của Docker

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg

sudo install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
\
  https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo $VERSION_CODENAME) stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Cài Docker Engine và Compose plugin

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Kiểm tra

```
docker --version
docker compose version
```

> **Kết quả mong đợi:**
>
> ```
> Docker version 24.x.x
> Docker Compose version v2.x.x
> ```

(Tùy chọn) Chạy Docker không cần sudo

```
sudo usermod -aG docker $USER
newgrp docker
```

3.10. Bước 6: Tạo docker-compose.yml trên VPS

Tạo thư mục làm việc trong home directory (không cần sudo):

```
mkdir -p ~/simple-reactjs
cd ~/simple-reactjs
nano docker-compose.yml
```

Nhập nội dung:

```
services:
  app:
    image: <your_dockerhub_username>/simple-reactjs:v1.0.0
    container_name: simple-reactjs
    ports:
      - "8081:80"
    restart: unless-stopped
```

Mở cổng 8081 trên firewall (nếu dùng `ufw`):

```
sudo ufw allow 8081
```

> **Lưu ý:** Map cổng `8081:80` — cổng 8081 của VPS tới cổng 80 bên trong container. Dùng cổng 8081 để tránh xung đột với các service khác đang chạy trên cổng 80.

3.11. Bước 7: Kéo image và khởi động container

```
cd ~/simple-reactjs
docker compose pull
docker compose up -d
```

Kiểm tra container đang chạy:

```
docker compose ps
```

> **Kết quả mong đợi:**
>
> ```
> NAME              STATUS     PORTS
> simple-reactjs    Up         0.0.0.0:8081->80/tcp
> ```

Mở trình duyệt và truy cập `http://<IP_VPS>:8081`.

> **Kết quả mong đợi:** Ứng dụng React hiển thị bình thường.

3.12. Cập nhật ứng dụng (khi có phiên bản mới)

Trên máy local:

```
# Build image mới với tag v1.1.0
docker build -t <your_dockerhub_username>/simple-reactjs:v1.1.0 .
docker push <your_dockerhub_username>/simple-reactjs:v1.1.0
```

Trên VPS, sửa file `docker-compose.yml` (đổi tag thành `v1.1.0`), rồi:

```
cd ~/simple-reactjs
docker compose pull
docker compose up -d
```

3.13. Các lệnh Docker quan trọng

Lệnh	Mô tả
`docker compose ps`	Xem trạng thái container
`docker compose logs -f`	Xem log container theo thời gian thực

Lệnh	Mô tả
`docker compose down`	Dừng và xóa container
`docker compose up -d`	Khởi động container ở chế độ nền
`docker images`	Xem danh sách image đã có
`docker rmi <image>`	Xóa image

3.14. Kết luận Lab 2

- **Multi-stage build** giúp image production nhỏ gọn và bảo mật hơn — chỉ chứa Nginx và file tĩnh
- VPS chỉ cần cài Docker, không cần cài Node.js hay cấu hình Nginx thủ công
- docker-compose giúp quản lý container đơn giản và dễ đọc hơn
- Hạn chế: mỗi lần cập nhật vẫn phải build và push thủ công

LAB 3. TỰ ĐỘNG HÓA CI/CD VỚI GITHUB ACTIONS

4.1. Mục tiêu

Sau lab này, học viên sẽ:

- Biết cài đặt và cấu hình **self-hosted runner** trong WSL (Windows Subsystem for Linux)
- Biết sử dụng **GHCR (GitHub Container Registry)** để lưu trữ Docker image — không cần tài khoản Docker Hub
- Hiểu sự khác biệt giữa GitHub-hosted runner và self-hosted runner
- Biết viết workflow GitHub Actions với `runs-on: self-hosted`
- Hiểu luồng tự động: push code → build image → push lên GHCR → deploy lên VPS
- Biết quy trình cập nhật và rollback phiên bản

4.2. Ý tưởng tổng thể

Hai điểm khác biệt chính so với hướng tiếp cận thông thường:

	Cách thông thường	Lab này
Registry	Docker Hub (bên thứ ba, cần account riêng)	**GHCR** — tích hợp sẵn trong GitHub, dùng `GITHUB_TOKEN` tự động
Runner	`ubuntu-latest` (máy ảo GitHub trả phí)	**Self-hosted trên WSL** — chạy ngay trên máy của học viên, miễn phí

Luồng hoạt động:

```
Máy Windows (WSL Runner)              GHCR                    VPS Ubuntu
────────────────────────              ────                    ──────────
git push origin main
        │
```

```
        │ GitHub Actions kích hoạt
        ▼
[WSL Runner nhận job]
  docker build .
  docker push ──────────────────▶ ghcr.io/owner/repo:sha
                                          │
                                [deploy job]
                                          │
                                  SSH vào VPS ──────────────▶ docker compose pull
                                                               docker compose up -d
```

Trước đây, mỗi lần cập nhật cần thực hiện thủ công:

> sửa code → commit → push → `docker build` → `docker push` → SSH vào VPS → `docker compose pull` → `docker compose up -d`

Với CI/CD: chỉ cần `git push`, mọi thứ còn lại tự động.

4.3. Bước 1: Cài đặt self-hosted runner trong WSL

Self-hosted runner là một chương trình cài trên máy của bạn, lắng nghe và thực thi các job từ GitHub Actions. Ở đây ta cài trong WSL Ubuntu để có môi trường Linux giống với production.

4.3.1. Mở WSL và chuẩn bị thư mục

Mở **Windows Terminal**, chọn tab Ubuntu (WSL), rồi chạy:

```
mkdir -p ~/actions-runner && cd ~/actions-runner
```

4.3.2. Lấy lệnh cài đặt runner từ GitHub

Vào repository trên GitHub → **Settings** → **Actions** → **Runners** → **New self-hosted runner**.

Chọn:

- **Operating System:** Linux
- **Architecture:** x64

GitHub sẽ hiển thị một bộ lệnh. Copy và chạy từng nhóm trong WSL:

Tải runner:

```
# Kiểm tra phiên bản mới nhất tại https://github.com/actions/runner/releases
# và thay 2.x.x bằng phiên bản hiện tại
curl -O -L https://github.com/actions/runner/releases/download/v2.x.x/actions-
runner-linux-x64-2.x.x.tar.gz
tar xzf ./actions-runner-linux-x64-2.x.x.tar.gz
```

> **Lưu ý:** Copy lệnh trực tiếp từ trang GitHub để có đúng phiên bản và checksum.

Cấu hình runner (dùng token từ trang GitHub):

Token xuất hiện trực tiếp trên trang `https://github.com/<OWNER>/simple-reactjs/settings/actions/runners`. Chạy:

```
./config.sh --url https://github.com/<OWNER>/simple-reactjs --token
<TOKEN_TU_GITHUB>
```

Khi được hỏi, nhập:

- **Runner name:** nhấn Enter để dùng tên mặc định (tên máy tính)
- **Runner labels:** nhập `wsl` (thêm label để dễ phân biệt)
- **Work folder:** nhấn Enter để dùng mặc định `_work`

> **Kết quả mong đợi:** Xuất hiện thông báo `√ Runner successfully added` và `√ Runner connection is good`.

4.3.3. Cài runner như một service hệ thống

```
sudo ./svc.sh install
sudo ./svc.sh start
```

Kiểm tra runner đang chạy:

```
sudo ./svc.sh status
```

> **Kết quả mong đợi:** Trạng thái `active (running)`.

Trên GitHub, vào **Settings** → **Actions** → **Runners** — runner của bạn sẽ hiển thị trạng thái **Idle** (sẵn sàng nhận job).

4.3.4. Đảm bảo Docker hoạt động trong WSL

```
docker --version
docker run hello-world
```

> **Nếu Docker không chạy được trong WSL:** Mở Docker Desktop trên Windows → Settings → Resources → WSL Integration → bật tích hợp cho distro Ubuntu của bạn → Apply & Restart.

4.3.5. Xóa runner và cài lại từ đầu

Nếu cần xóa hoàn toàn runner để cài lại (sai cấu hình, đổi repo, đổi label...), thực hiện theo thứ tự:

Bước 1: Dừng và gỡ service

```
cd ~/actions-runner
sudo ./svc.sh stop
sudo ./svc.sh uninstall
```

Bước 2: Xóa đăng ký runner khỏi GitHub

```
./config.sh remove --token <REMOVE_TOKEN>
```

Lấy `<REMOVE_TOKEN>`: vào GitHub → **Settings** → **Actions** → **Runners** → click vào runner → **Remove runner** → GitHub hiển thị token dùng một lần.

> Nếu muốn xóa nhanh mà không cần token (runner đã offline), vào GitHub → **Settings** → **Actions** → **Runners** → click runner → **Remove runner** → xác nhận. Runner bị xóa khỏi GitHub dù máy local chưa chạy lệnh `config.sh remove`.

Bước 3: Xóa thư mục runner

```
cd ~
rm -rf ~/actions-runner
```

Bước 4: Cài lại từ đầu

Tạo lại thư mục và làm lại từ mục 4.3.1:

```
mkdir -p ~/actions-runner && cd ~/actions-runner
```

Vào GitHub → **Settings** → **Actions** → **Runners** → **New self-hosted runner** để lấy token cài đặt mới.

4.4. Bước 2: Cấu hình quyền GHCR

4.4.1. Bật Workflow write permissions

Vào repository → **Settings** → **Actions** → **General** → **Workflow permissions** → Chọn **"Read and write permissions"** → **Save**.

Bước này cho phép `GITHUB_TOKEN` được cấp quyền `write:packages` để push image lên GHCR. Không cần tạo secret nào thêm.

> `GITHUB_TOKEN` **không cần tạo thủ công** — GitHub tự động cung cấp trong mỗi workflow run. Chỉ cần khai báo `permissions: packages: write` trong job là đủ.

4.4.2. Cấu hình visibility của package GHCR (lần đầu push)

Lần đầu push image, GitHub tạo package với visibility **private** mặc định. Để kiểm tra hoặc pull từ bên ngoài workflow, có thể đặt thành **Public**:

Vào GitHub → tab **Packages** → chọn package `simple-reactjs` → **Package Settings** → **Change visibility** → **Public**.

> Nếu giữ private, image vẫn được pull trong workflow nhờ `GITHUB_TOKEN`. Chỉ cần thêm bước khi muốn pull từ máy khác.

4.5. Bước 3: Tạo workflow CI/CD (Build & Push)

Trong thư mục dự án, tạo file:

```
mkdir -p .github/workflows
```

Tạo file `.github/workflows/build.yml`:

```
name: Build and Push GHCR Image

on:
  push:
    branches: [ "main" ]

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository_owner }}/simple-reactjs

jobs:
  build-and-push:
    name: Build & Push to GHCR
    runs-on: [self-hosted, wsl]
    permissions:
      contents: read
      packages: write

    steps:
      - name: Lấy source code
        uses: actions/checkout@v4

      - name: Lowercase image name
        run: echo "IMAGE_NAME=$(echo '${{ env.IMAGE_NAME }}' | tr '[:upper:]'
'[:lower:]')" >> $GITHUB_ENV

      - name: Configure Docker credentials store
```

```
          run: |
            mkdir -p ~/.docker
            echo '{}' > ~/.docker/config.json

        - name: Đăng nhập GitHub Container Registry
          uses: docker/login-action@v3
          with:
            registry: ghcr.io
            username: ${{ github.actor }}
            password: ${{ secrets.GITHUB_TOKEN }}

        - name: Set up Docker Buildx
          uses: docker/setup-buildx-action@v3

        - name: Build và Push Docker Image
          uses: docker/build-push-action@v6
          with:
            context: .
            push: true
            provenance: false
            tags: |
              ghcr.io/${{ env.IMAGE_NAME }}:latest
              ghcr.io/${{ env.IMAGE_NAME }}:${{ github.sha }}
```

> **Lưu ý về username chữ hoa:** GHCR yêu cầu image name phải viết **thường hoàn toàn**. Nếu `github.repository_owner` có chữ hoa (ví dụ `GiangLT-dau`), bước `Lowercase image name` sẽ chuyển về `gianglt-dau` trước khi push/pull. Thiếu bước này sẽ gây lỗi `denied` hoặc `not found` trên GHCR.

4.6. Giải thích workflow

Trigger:

```
on:
  push:
    branches: [ "main" ]
```

Workflow tự động chạy mỗi khi có commit push lên nhánh `main`.

`runs-on: [self-hosted, wsl]`:

Chỉ định job chạy trên runner có cả hai label `self-hosted` và `wsl` — tức runner WSL của bạn đã gắn label ở bước 4.3.2.

`permissions: packages: write`:

Cho phép `GITHUB_TOKEN` được cấp quyền ghi lên GHCR trong phạm vi job này. Không cần secret bổ sung.

`docker/setup-buildx-action`:

Kích hoạt BuildKit — backend build hiện đại của Docker, hỗ trợ cache layer, multi-platform. Cần thiết khi dùng `docker/build-push-action`.

`docker/build-push-action`:

Tham số	Giá trị	Giải thích
`context: .`	Thư mục hiện tại	Build từ source code trong repo
`push: true`	Bật	Push image lên registry sau khi build
`provenance: false`	Tắt	Tắt SBOM attestation — tránh lỗi manifest trên một số registry
`tags`	2 tag	`latest` dễ pull; SHA để truy vết và rollback

Ưu điểm GHCR so với Docker Hub:

- Không cần tài khoản bên thứ ba
- `GITHUB_TOKEN` tự động, không quản lý thêm credentials
- Image cùng namespace với source code, quản lý permissions đồng nhất
- Miễn phí cho public repository

4.7. Chạy container trực tiếp trên WSL (kiểm tra nhanh)

Sau khi workflow chạy thành công, có thể pull và chạy image ngay trong WSL để xác nhận:

```
docker pull ghcr.io/<GITHUB_USERNAME>/simple-reactjs:latest
docker run -d -p 3000:80 --name test-reactjs ghcr.io/<GITHUB_USERNAME>/simple-reactjs:latest
```

Mở trình duyệt tại `http://localhost:3000`.

> **Kết quả mong đợi:** Ứng dụng React hiển thị bình thường.

Dừng container test:

```
docker stop test-reactjs && docker rm test-reactjs
```

4.8. Bước 4: Commit và push workflow lên GitHub

```
git add .
git commit -m "Add CI/CD workflow with GHCR and WSL runner"
git push origin main
```

> **Kết quả mong đợi:** Tab **Actions** trên GitHub hiển thị workflow đang chạy, runner WSL nhận job.

4.9. Bước 5: Theo dõi pipeline

Trên GitHub, vào tab **Actions** → chọn workflow đang chạy.

Quan sát từng bước:

```
✅ Lấy source code
✅ Đăng nhập GitHub Container Registry
✅ Set up Docker Buildx
✅ Build và Push Docker Image   ← thường mất 1–3 phút
```

Sau khi tất cả xanh, kiểm tra image trên GitHub → tab **Packages** → `simple-reactjs`. Image phải có 2 tag: `latest` và hash SHA commit.

4.10. Bước 6: Kiểm tra luồng CI/CD hoạt động

Sửa nội dung trong `src/App.jsx`:

```jsx
// Thêm một dòng text bất kỳ để kiểm tra
<p>Version 2 — CI/CD with GHCR is working!</p>
```

Sau đó:

```
git add .
git commit -m "Test CI/CD pipeline with GHCR"
git push origin main
```

Chờ workflow chạy xong → tab **Packages** sẽ có tag SHA mới.

4.11. Tối ưu thêm: Chỉ build khi code đã qua kiểm tra

Thêm bước lint trước khi build (cập nhật job `build-and-push` trong `build.yml`):

```yaml
    steps:
      - name: Lấy source code
        uses: actions/checkout@v4

      - name: Cài Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '22'
          cache: 'npm'
```

```yaml
      - name: Cài dependency
        run: npm ci

      - name: Kiểm tra Lint
        run: npm run lint

      - name: Đăng nhập GitHub Container Registry
        uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}

      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v3

      - name: Build và Push Docker Image
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          provenance: false
          tags: |
            ghcr.io/${{ env.IMAGE_NAME }}:latest
            ghcr.io/${{ env.IMAGE_NAME }}:${{ github.sha }}
```

> Nếu `npm run lint` thất bại, workflow dừng ngay — không build, không push.

4.12. Kết luận Lab 3

- **Self-hosted WSL runner** giúp job chạy ngay trên máy của bạn — không tốn phí, không bị giới hạn thời gian, tận dụng Docker đã cài sẵn
- **GHCR** tích hợp liền mạch với GitHub: dùng `GITHUB_TOKEN` tự động, không cần tài khoản bên thứ ba
- Dùng `github.sha` làm tag image để truy vết phiên bản và hỗ trợ rollback
- Workflow này tập trung vào **build & push** — phần deploy lên VPS xem thêm ở phần mở rộng bên dưới

---

MỞ RỘNG LAB 3: TỰ ĐỘNG DEPLOY LÊN VPS

> Phần này là **tùy chọn**, dành cho học viên muốn tự động hóa cả bước deploy lên VPS sau khi image đã được push lên GHCR.

E.1. Ý tưởng

Sau khi `build.yml` push image lên GHCR thành công, một workflow riêng (`deploy.yml`) sẽ SSH vào VPS, kéo image mới và khởi động lại container — hoàn toàn tự động.

Hai workflow tách biệt:

```
build.yml          →  Build & Push image lên GHCR  (WSL runner)
deploy.yml         →  SSH vào VPS, docker compose up  (WSL runner)
```

Dùng sự kiện `workflow_run` để `deploy.yml` chỉ chạy khi `build.yml` hoàn thành thành công.

E.2. Cấu hình Secrets cho VPS

Vào repository → **Settings** → **Secrets and variables** → **Actions** → **New repository secret**.

Secret	Giá trị	Mục đích
`VPS_HOST`	Địa chỉ IP VPS	SSH vào VPS khi deploy
`VPS_USERNAME`	`root` hoặc user VPS	SSH username
`VPS_PASSWORD`	Mật khẩu VPS	SSH password

E.3. Chuẩn bị VPS

SSH vào VPS và tạo thư mục làm việc, cấp quyền cho user hiện tại:

```
sudo mkdir -p /opt/simple-reactjs
sudo chown $USER:$USER /opt/simple-reactjs
cd /opt/simple-reactjs
```

> **Giải thích:** `chown` chuyển quyền sở hữu về user hiện tại để các bước sau (tạo file, chỉnh sửa) không cần `sudo`.

Tạo `docker-compose.yml` trỏ sang GHCR, dùng cổng **8081** để tránh xung đột với các service khác đang chạy trên cổng 80:

```
cat > docker-compose.yml << 'EOF'
services:
  app:
    image: ghcr.io/<GITHUB_USERNAME>/simple-reactjs:latest
    container_name: simple-reactjs
    ports:
      - "8081:80"
    restart: unless-stopped
EOF
```

> Thay `<GITHUB_USERNAME>` bằng username GitHub của bạn (chữ thường), ví dụ: `ghcr.io/gianglt-dau/simple-reactjs:latest`

Mở cổng 8081 trên firewall (nếu dùng `ufw`):

```
sudo ufw allow 8081
```

Nếu package GHCR để **private**, đăng nhập một lần trên VPS:

```
# Tạo PAT với scope read:packages trên GitHub trước
echo "<PAT_TOKEN>" | docker login ghcr.io -u <GITHUB_USERNAME> --password-stdin
```

Nếu package để **public** (xem mục 4.4.2), bỏ qua bước này.

E.4. Tạo workflow deploy.yml

Tạo file `.github/workflows/deploy.yml`:

```
name: Deploy to VPS

on:
  workflow_run:
    workflows: ["Build and Push GHCR Image"]
    types:
      - completed

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: ${{ github.repository_owner }}/simple-reactjs

jobs:
  deploy:
    name: Deploy to VPS
    if: ${{ github.event.workflow_run.conclusion == 'success' }}
    runs-on: [self-hosted, wsl]
    permissions:
      contents: read
      packages: read

    steps:
      - name: Lowercase image name
        run: echo "IMAGE_NAME=$(echo '${{ env.IMAGE_NAME }}' | tr '[:upper:]'
'[:lower:]')" >> $GITHUB_ENV

      - name: Configure Docker credentials store
        run: |
          mkdir -p ~/.docker
          echo '{}' > ~/.docker/config.json

      - name: SSH vào VPS và Deploy
        uses: appleboy/ssh-action@v1.0.3
        with:
```

```
            host: ${{ secrets.VPS_HOST }}
            username: ${{ secrets.VPS_USERNAME }}
            password: ${{ secrets.VPS_PASSWORD }}
            script: |
              cd /opt/simple-reactjs

              # Cập nhật tag image theo SHA commit hiện tại
              SHA=${{ github.event.workflow_run.head_sha }}
              sed -i "s|image:.*|image: ghcr.io/${{ env.IMAGE_NAME }}:${SHA}|g"
  docker-compose.yml

              # Kéo image mới và khởi động lại container
              docker compose pull
              docker compose up -d

              # Dọn dẹp image cũ không dùng
              docker image prune -f
```

Giải thích điểm quan trọng:

Dòng	Giải thích
`workflow_run: workflows: ["Build and Push GHCR Image"]`	Lắng nghe khi workflow có tên đó kết thúc
`if: conclusion == 'success'`	Chỉ deploy khi build thành công, không deploy khi build lỗi
`github.event.workflow_run.head_sha`	SHA của commit đã trigger build — dùng làm tag image để khớp chính xác
`packages: read`	Quyền đọc GHCR cho `GITHUB_TOKEN` trong job này

E.5. Theo dõi và xác nhận

Sau khi push code:

1. `build.yml` chạy trên WSL runner → build & push image lên GHCR
2. Khi `build.yml` hoàn thành → `deploy.yml` tự động kích hoạt
3. `deploy.yml` SSH vào VPS, kéo image mới, restart container

Trên GitHub tab **Actions**: bạn sẽ thấy 2 workflow riêng biệt chạy tuần tự.

Truy cập `http://<IP_VPS>:8081` để kiểm tra kết quả.

E.6. Rollback khi bản mới có lỗi

Lấy SHA của commit ổn định từ tab **Actions** trên GitHub. SSH vào VPS:

```
cd /opt/simple-reactjs

# Thay <SHA_CU> bằng SHA commit ổn định
sed -i "s|image:.*|image: ghcr.io/<GITHUB_USERNAME>/simple-reactjs:<SHA_CU>|g"
docker-compose.yml
docker compose pull
docker compose up -d
```

> Truy cập `http://<IP_VPS>:8081` để xác nhận phiên bản đã rollback.

PHẦN 5. TỔNG HỢP KIẾN THỨC SAU 3 LAB

5.1. So sánh 3 cách triển khai

	Deploy thủ công	**Docker**	**CI/CD (GHCR + WSL)**
Ưu điểm	Dễ hiểu, phù hợp khi mới bắt đầu	Môi trường nhất quán, dễ versioning, VPS sạch	Tự động hóa hoàn toàn, nhanh, ít rủi ro sai sót
Nhược điểm	Cập nhật thủ công, dễ sai	Cần học Docker, cần build và push thủ công	Cần cài runner, WSL phải chạy liên tục khi có push
Registry	Không có	Docker Hub	**GHCR** — tích hợp GitHub, dùng `GITHUB_TOKEN`
Runner	Không có	Không có	**Self-hosted WSL** — chạy trên máy local, miễn phí
Thời gian deploy	~5 phút	~3 phút	~2–3 phút (tự động)
Phù hợp với	Học tập, thử nghiệm nhanh	Cá nhân/team nhỏ	Team, môi trường production

5.2. Luồng dữ liệu qua 3 lab

```
Máy local (Windows/WSL)       GHCR                    VPS Ubuntu
────────────────────────      ────                    ──────────

source code (trên GitHub)
    │
    │  git clone + npm run build (Lab 1)
    ▼
  dist/  ─── cp -r dist/* ──────────────────────▶ /var/www/
    │
    │  docker build (Lab 2)
    ▼
  image ──── docker push ──▶ Docker Hub ── pull ──▶ container
    │
    │  git push (Lab 3 — WSL Runner)
```

```
    ▼
GitHub Actions
[WSL Runner]                                        SSH deploy
  docker build .                                         │
  docker push ──────────▶ ghcr.io/owner/repo:sha         │
                                  │                      │
                                  └──── docker pull ─────▶ container
```

PHẦN 6. BÀI TẬP THỰC HÀNH ĐỀ XUẤT

Bài tập 1 (Lab 1): Cập nhật nội dung trong `src/App.jsx`, chạy `git pull` trên VPS, build lại và kiểm tra trang web đã cập nhật. Kiểm tra xem URL trực tiếp (`http://<IP_VPS>/about`) có hoạt động không.

Bài tập 2 (Lab 2): Chỉnh `docker-compose.yml` để map cổng `8082:80` thay vì `8081:80`. Truy cập bằng `http://<IP_VPS>:8082`.

Bài tập 3 (Lab 2): Thêm tag `latest` khi push image lên Docker Hub bằng tay:

```
docker tag <user>/simple-reactjs:v1.0.0 <user>/simple-reactjs:latest
docker push <user>/simple-reactjs:latest
```

Bài tập 4 (Lab 3): Thêm bước `npm run lint` vào workflow trước khi build. Cố tình tạo lỗi lint và quan sát workflow dừng ở đâu.

Bài tập 5 (Lab 3): Cố tình sửa code làm app hiển thị sai, push lên và thực hành rollback về SHA commit trước đó bằng lệnh `sed` trực tiếp trên VPS.

Bài tập 6 (Lab 3): Dừng WSL runner (`sudo ./svc.sh stop`), push một commit, quan sát job ở trạng thái `Queued`. Khởi động lại runner và xem job tự chạy.

PHẦN 7. LỖI THƯỜNG GẶP VÀ CÁCH XỬ LÝ

Lỗi	Nguyên nhân	Cách xử lý
Trang trắng khi truy cập VPS	File build chưa được upload đúng thư mục	Kiểm tra `ls /var/www/simple-reactjs/`, phải có `index.html`
404 khi refresh trang hoặc vào URL trực tiếp	Nginx chưa có `try_files $uri $uri/ /index.html`	Thêm dòng này vào cấu hình Nginx, reload nginx
`nginx -t` báo lỗi cú pháp	Sai cú pháp trong file cấu hình	Đọc kỹ thông báo lỗi, kiểm tra lại dấu `;` và `{}`
`docker build` thất bại	Lỗi trong source code hoặc `npm ci` thất bại	Đọc log build, kiểm tra `package.json` và `package-lock.json`
`docker push` thất bại	Chưa đăng nhập hoặc sai tên image	Chạy `docker login` lại, kiểm tra tên image có đúng format `username/repo:tag`

Lỗi	Nguyên nhân	Cách xử lý
GitHub Actions fail tại bước login GHCR	`GITHUB_TOKEN` thiếu quyền	Kiểm tra `permissions: packages: write` đã khai báo trong job chưa
`error storing credentials - fork/exec docker-credential-desktop.exe: exec format error`	WSL runner kế thừa Docker config của Windows Desktop, có `credsStore: desktop` trỏ tới file `.exe` không chạy được trên Linux	Thêm bước trước khi login: `mkdir -p ~/.docker && echo '{}' > ~/.docker/config.json` để xóa credential store
Push/pull GHCR bị lỗi `denied` hoặc `not found` dù đã login thành công	GitHub username chứa chữ hoa; GHCR yêu cầu image name phải hoàn toàn **lowercase**	Thêm bước lowercase trước khi login: `echo "IMAGE_NAME=$(echo '${{ env.IMAGE_NAME }}' | tr '[:upper:]' '[:lower:]')" >> $GITHUB_ENV`
Job ở trạng thái `Queued` mãi không chạy	WSL runner không hoạt động	Vào WSL, chạy `sudo ./svc.sh status`; nếu stopped thì `sudo ./svc.sh start`
GitHub Actions fail tại bước SSH	Thông tin VPS sai hoặc VPS chặn SSH bằng password	Kiểm tra `VPS_HOST`, `VPS_USERNAME`, `VPS_PASSWORD`; đảm bảo VPS cho phép `PasswordAuthentication yes` trong `/etc/ssh/sshd_config`
VPS không pull được image từ GHCR	Package private, VPS chưa xác thực	Đặt package thành public trên GitHub, hoặc chạy `docker login ghcr.io` trên VPS với PAT
Container chạy nhưng web không hiển thị	Cổng chưa được mở trên firewall VPS	Mở đúng cổng đang dùng: `ufw allow 8081` (Lab 2/3 dùng 8081) hoặc cấu hình Security Group trên cloud provider